

src/app/codeCityRenderer.ts

Kind: added · Baseline: 9aadd5eb5604 · Target: NovaDiff · Generated: 2026-05-26T19:28:20.117Z

Overview

`src/app/codeCityRenderer.ts` adds a renderer factory for the Code City visualizer. It exports a `CityRenderer` interface, a `CityRendererBackend` union type, and an async `createCityRenderer` that chooses WebGPU when available, otherwise WebGL.

Key changes

- **R1:** `import * as THREE from "three"` – pulls in Three.js core.
- **R3:** `export type CityRendererBackend = "webgpu" | "webgl"` – declares supported backends.
- **R5:** `export interface CityRenderer` – exposes `renderer: THREE.WebGLRenderer`, `backend: CityRendererBackend`, and `dispose(): void`.
- **R11:** `export interface CreateCityRendererOptions` – specifies canvas, size, pixel ratio, alpha, antialias, and `preferWebGpu`.
- **R26:** `export async function createCityRenderer(options: CreateCityRendererOptions)` –
 - attempts a dynamic import of `three/webgpu`; on success it creates a `WebGPURenderer`, initializes it, sets pixel ratio, size, clear color, and color space, then returns a `CityRenderer` with `backend: "webgpu"`.
 - if the import fails or WebGPU is unavailable, it falls back to a standard `THREE.WebGLRenderer` with tone mapping and exposure settings, returning `backend: "webgl"`.

Impact

- **Reliability:** Guarantees a renderer instance is returned, with a graceful fallback.
- **Maintainability:** Centralizes backend selection, simplifying future changes.
- **Observability:** Emits a console warning when WebGPU cannot be used (see R60-61).

Risks & follow-ups

- **Dynamic import failure** – verify that `three/webgpu` resolves in all target browsers; test on environments lacking WebGPU.
- **Pixel ratio handling** – ensure `options.pixelRatio` is applied in both backends (R47, R68).
- **Tone mapping exposure** – WebGL path sets `toneMappingExposure = 1.05` (R72); confirm visual consistency.
- **Cleanup** – `dispose()` calls `renderer.dispose()`; check for residual references to avoid memory leaks.